



Murakkab: Resource-Efficient Agentic Workflow Orchestration in Cloud Platforms

OSDI '26

Gohar Irfan Chaudhry¹, Esha Choukse², Haoran Qiu², Íñigo Goiri²,
Rodrigo Fonseca², Adam Belay¹, Ricardo Bianchini³



¹ MIT CSAIL



² Microsoft Azure Research / ³ Microsoft Azure

Presenter: Yunpeng Xu

2026.7.27

Author & Team



Lead author

Gohar Irfan Chaudhry

MIT EECS PhD Student

MIT CSAIL / PDOS

Microsoft Research Systems (3 年)
云平台与 Serverless 效率研究

研究方向

云系统和 AI 服务的资源效率

Runtime, OS and HW/SW co-design

[Homepage](#)

[Google Scholar](#)

从云系统资源效率走向 Agentic Workflow 端到端优化

同一研究主线贯穿云运行时、操作系统、数据中心资源管理与 Compound AI serving。



Prof. Adam Belay

PhD advisor, MIT CSAIL / PDOS

OS、runtime 与 distributed systems
面向更快、更高效、更安全的数据中心

[Adam Belay](#) / [MIT PDOS](#)



Azure Research - Systems

云与 AI 基础设施效率
资源优化、可持续性与技术落地 Azure
[Azure Systems Research](#)

近期代表作 (Gohar)

[OSDI '26](#) Murakkab: Resource-Efficient Agentic Workflow Orchestration (第一作者)

[NSDI '26](#) Sandook: Taming Datacenter SSD Performance Variability (第一作者)

[HotOS '25](#) Towards Resource-Efficient Compound AI Systems (第一作者)

[NSDI '24](#) Making Kernel Bypass Practical for the Cloud with Junction (第二作者)

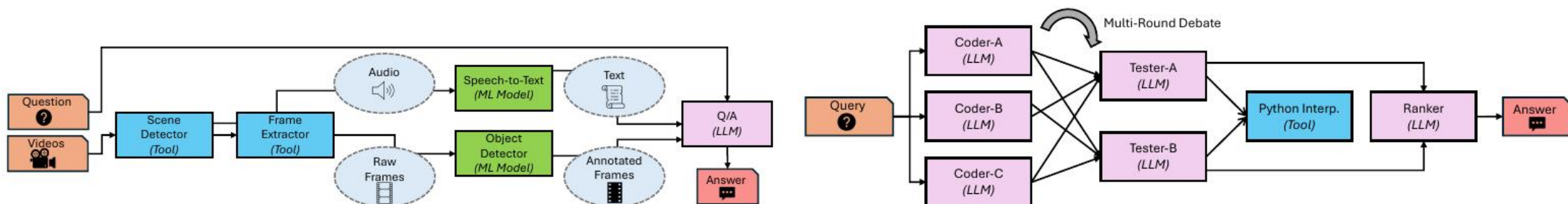
研究脉络: Cloud runtime / resource efficiency → Compound AI → Agentic workflow orchestration

Contents

- **Background & Motivation**
- **Challenges**
- **Design**
- **Evaluation**
- **Thinking**

Background & Motivation

Agentic workflow 让一次请求从单次模型调用，变成多组件调用图



(a) Video Q/A workflow [87]: multi-modal with tools handling different modalities before feeding into an LLM for answering the query.

(b) Code generation workflow [25, 55]: text-only with an LLM Debate structure to write, test and execute code.

Figure 1: Two agentic workflows with different characteristics and components.

- (a) Video Q/A: 场景检测后，音频转写与视频帧处理并行 → 多模态 LLM 汇合作答
- (b) Code Generation: 多个 Coder ↔ Tester 多轮 debate → Python 解释器 → Ranker 选优

一次请求的精度、延迟、能耗与成本，取决于整张图的配置，而不是其中某一次调用
而三方目标并不一致：开发者关心质量与延迟，终端用户额外关心成本，平台关心资源效率与成本

Background & Motivation

一次请求的系统路径：六层抽象，分属不同控制主体

背景调研：先厘清各层管理的对象与责任归属，再理解 **Murakkab** 的跨层优化范围

请求入口 Query / Event + request SLO (质量、时延、成本、隐私)			
层次	信息 / 控制对象	本层需要回答的问题	典型字段 / 执行对象
L1	业务语义	结果何时正确，哪些行为不可接受？	goal、correctness、permission、SLO
L2	Workflow	任务如何分解、依赖、分支与终止？	nodes、edges、branch、termination
L3	Agent 能力绑定	每个节点允许由哪些模型、工具或服务完成？	model、tool、MCP / service、params
L4	应用运行时	本次请求执行到哪里，下一步谁就绪？	session、state、ready node、trace
L5	资源 / 推理配置	用什么后端与容量承载这些组件？	backend、quantization、replicas、capacity
L6	请求级调度	本次组件调用发送到哪个实例？	endpoint、queue / batch、device、placement

App Developer / 应用团队
定义业务语义、workflow 与合法候选空间

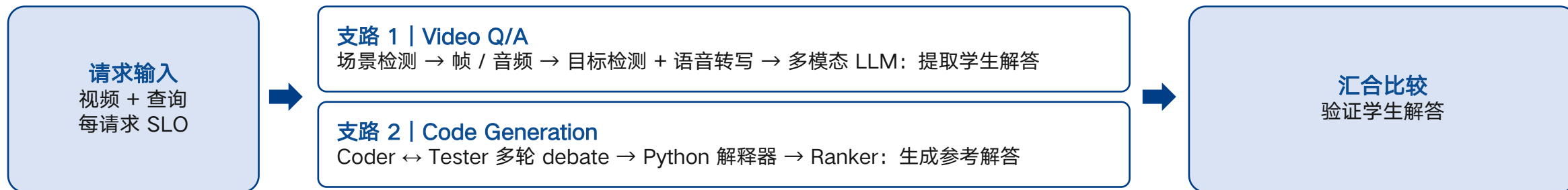
L4 - L5: 部署形态决定责任
Managed / 托管：平台
Self-hosted / BYOM：应用

Serving / Scheduler
实例、batch 与 placement

六层不是六个串行阶段，而是同一条任务在不同抽象层上的信息与控制边界。
Murakkab 关注 L2 / L3 / L5 的联合配置；L4 推进可执行 workflow，L6 落实每次 dispatch。

Background & Motivation

示例请求: **Verify the student's coding solution from a video**, latency SLO = 30 秒



Orchestrator 为该请求构造带 fan-out 的 DAG: 两条子任务可并行, 完成后汇合比较



这一个请求同时牵动 Workflow / Agent / Hardware 三层

抽帧数改变 token 量, token 量进一步影响延迟与所需加速器;
任何一层单独调优, 都无法保证端到端的精度、延迟、能耗与成本

Challenges

Insight 1 | 平台看不见 workflow 内部，开发者缺少系统级资源控制与洞察

```
1 # == Workflow nodes (coupled app logic + configs) ==
2 scene_detection = Tool(
3     fn=SceneDetector(), # Implemented earlier by the
4     key=AWS_KEY, resources={"CPUs": 32}
5 )
6 frame_extractor = Tool(
7     fn=FrameExtractor(), # Implemented earlier by the
8     key=AWS_KEY, params={"num_frames": 15}, resources={"CPUs": 32}
9 )
10 speech_to_text = MLModel(
11     name="Whisper", key=OPENAI_API_KEY, resources={"PTUs": 50}
12 )
13 object_detection = MLModel(
14     name="CLIP", key=AZURE_KEY, resources={"CPUs": 128}
15 )
16 question_answer = LLM(
17     name="Llama-3.2", key=DATABRICKS_API_KEY,
18     params={"batch": 256}, resources={"GPUs": 8, "Type": "H100"},
19     system_prompt="You are an agent that can understand videos.",
20     user_prompt="Answer the given question about the video."
21 )
22 # == Query and data ==
23 ques = "What is the name of the person wearing the red dress?"
24 videos = ["road_trip.mp4"]
25 # == Workflow (ordering of nodes and data flow) ==
26 scenes, audio = scene_detection(videos)
27 frames = frame_extractor(scenes)
28 transcript = speech_to_text(audio)
29 obj_frames = object_detection(frames)
30 answer = question_answer(ques, transcript, obj_frames)
```

Listing 1: Simplified video Q/A workflow definition. It shows tightly coupled application logic and execution details typical of existing frameworks.

同一份 workflow 定义中紧耦合的三类配置

Workflow

节点顺序与数据流；是否启用 speech-to-text / object detection

Agent

FrameExtractor 帧数；各组件使用的模型与 LLM batch

Hardware

CPU / PTU / GPU 数量，以及 GPU 类型

双方各自掌握半张图，缺少共同的决策面

开发者 / Agent 框架

掌握： workflow 组件、执行逻辑、交互关系与配置参数

缺少： 对系统级资源行为的控制或洞察

云平台 / Serving

关注： 资源效率与成本；模型部署与 GPU 配额等平台约束

缺少： workflow 组件 / 模型、任务序列、数据流与任务要求



平台看不到任务、依赖与端到端要求，开发者看不到系统级资源行为；
两侧信息无法汇合，整条 workflow 难以进行端到端优化。

Challenges

Insight 2 | 多层旋钮共同构成精度、延迟、能效与成本的多目标权衡

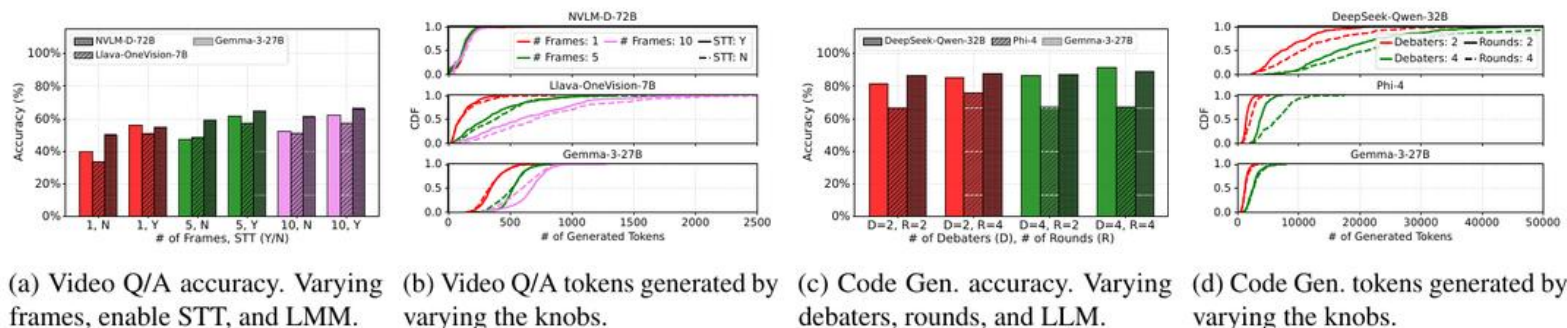


Figure 2: Workflow accuracy under different configurations and the token generation load on the respective models.

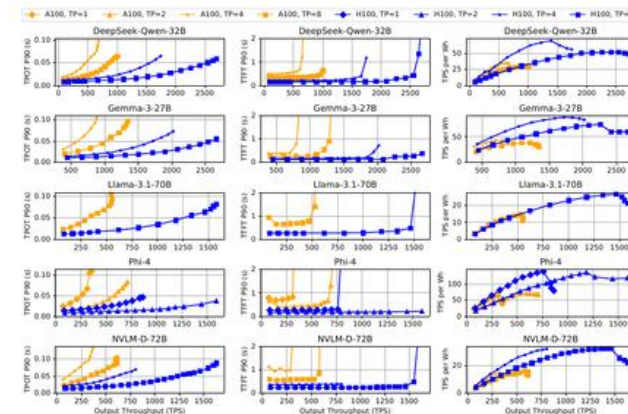


Figure 3: Model performance under different hardware and parallelism configurations.

Workflow / Agent 层旋钮 → accuracy vs. latency / cost

- Video Q/A: Gemma-3-27B + 10 frames + STT 达到 66.2% accuracy, 并在该模型展示的配置中生成 token 最多。
- Code Generation: 相同 workflow 配置下, DeepSeek-Qwen-32B 的中位 token 约 20,000, Gemma-3-27B 约 2,500。

Hardware 层旋钮 → cost vs. performance

A100 / H100 与不同 tensor parallelism 在不同请求负载下, 呈现不同的 P90 TTFT、P90 TPOT、throughput 与 TPS/Wh。

Workflow / Agent knobs 形成 accuracy 与 latency / cost 权衡, Hardware knobs 形成 cost 与 performance 权衡; 多目标优化需要跨层联合决策。

Challenges

Insight 3 | 少量跨层 knobs 组合后，配置空间即呈组合式增长

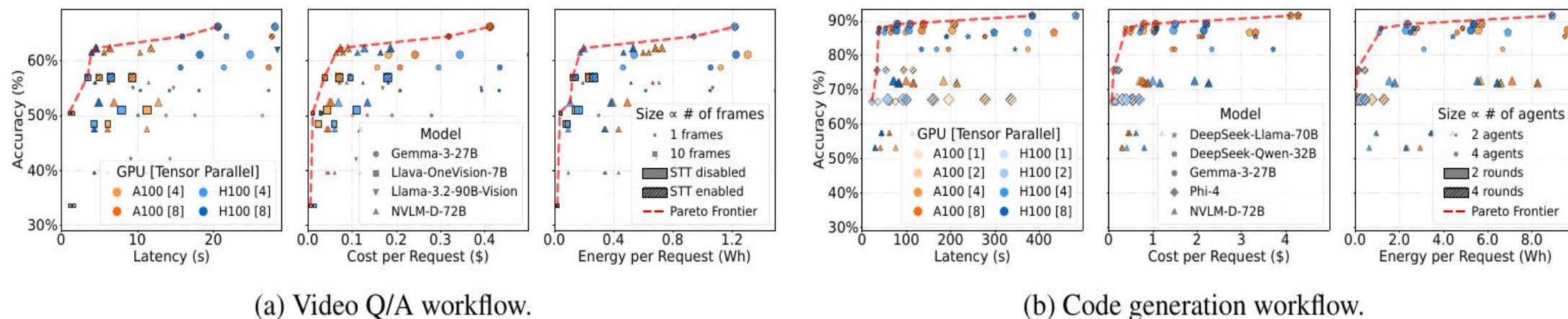


Figure 4: Large space of workflow configurations along a subset of knobs and metrics.

01 用户目标不同

不同用户分别关注低延迟、低成本或高准确率。

02 运行时上下文变化

最优配置取决于用户流量、模型可用性与云资源可用性。

03 模型与硬件快速演进

可用模型与硬件选择持续快速变化。

配置空间随三层 knobs 组合增长: $O(\#WorkflowKnobs \times \#AgentKnobs \times \#HardwareKnobs)$;
动态目标与运行时条件使手工导航难以持续。

Challenges

三个 Insight 共同指向：系统必须同时具备三项能力

01

Workflow visibility

看见 workflow 的任务、依赖、
执行约束与 SLO

02

Holistic control

整体控制 workflow 的
开发与执行栈

03

Automated configuration

根据 SLO 与运行时条件
自动生成并调整配置

三项能力共同构成 Murakkab 的端到端优化基础

三项能力如何落实到 Development → Optimization & Deployment → Execution?

Design

Murakkab 系统架构：贯穿 workflow 开发、优化部署与执行

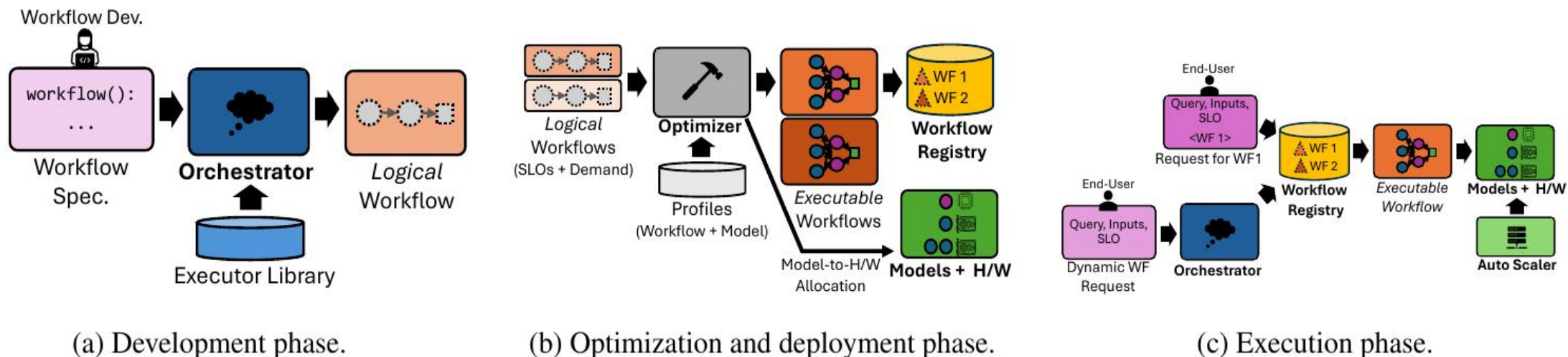


Figure 5: Murakkab manages end-to-end workflow life-cycle: from development to optimized deployment and execution.

01 Development

Workflow Spec → initial executor assignment → request-agnostic Logical Workflow

02 Optimization & Deployment

Profiles + SLO tiers + demand
→ Executable Workflow + deployment plan

03 Execution

Registry lookup → per-request dispatch
Auto-Scaler 持续扩缩容

Logical Workflow 连接开发与优化，Workflow Registry 连接部署计划与运行时执行。

Design

Workflow visibility: 开发者声明任务与数据流, 低层配置交给后续阶段

对应 Challenge 01

Imperative: 任务逻辑与执行配置耦合

```
1 # == Workflow nodes (coupled app logic + configs) ==
2 scene_detection = Tool(
3     fn=SceneDetector(), # Implemented earlier by the
4     key=AWS_KEY, resources={"CPUs": 32}
5 )
6 frame_extractor = Tool(
7     fn=FrameExtractor(), # Implemented earlier by the
8     key=AWS_KEY, params={"num_frames": 15}, resources={"CPUs": 32}
9 )
10 speech_to_text = MLModel(
11     name="Whisper", key=OPENAI_API_KEY, resources={"PTUs": 50}
12 )
13 object_detection = MLModel(
14     name="CLIP", key=AZURE_KEY, resources={"CPUs": 128}
15 )
16 question_answer = LLM(
17     name="Llama-3.2", key=DATABRICKS_API_KEY,
18     params={"batch": 256}, resources={"GPUs": 8, "Type": "H100"},
19     system_prompt="You are an agent that can understand videos.",
20     user_prompt="Answer the given question about the video."
21 )
```

model / num_frames / batch / GPU 等配置与应用代码
共同演化

Declarative Workflow Specification

```
1 # == Sub-tasks in the workflow ==
2 scene_detect = "Given a list of videos, identify scenes in each."
3 frame_extract = "Given a list of scenes, extract frames."
4 stt = "Given a list of scenes, convert audio to text."
5 q_a = "Answer the query given some context."
6 # == Workflow description (sub-tasks and data flow) ==
7 def workflow(query, videos):
8     scenes = scene_detect(videos)
9     frames = frame_extract(scenes)
10    transcript = stt(scenes)
11    answer = q_a(query, [frames, transcript])
12    return answer
13 # == Execution with example request ==
14 query = "What is the name of the person wearing the red dress?"
15 videos = ["road_trip.mp4"]
16 result = run(workflow(query, videos), slo=LOW_LATENCY)
```

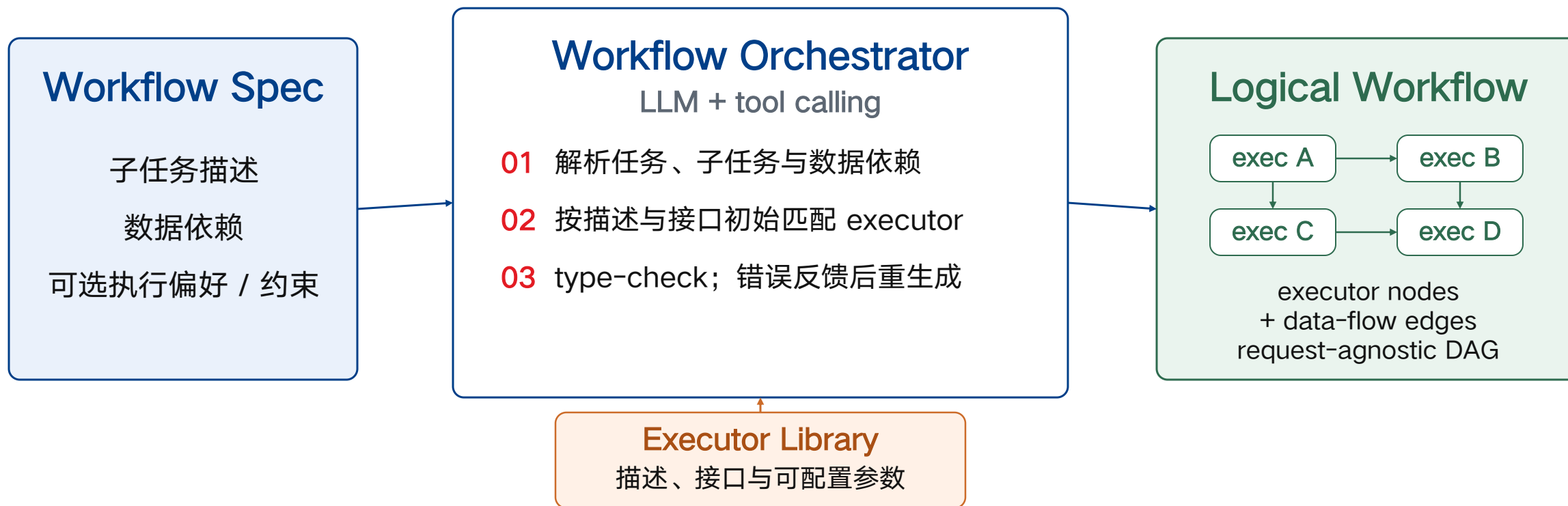
Lines 1 – 12 | 任务、子任务与数据依赖
Lines 13 – 16 | query + input payload + per-request SLO

Logical Workflow 不包含 query、input 或 per-request SLO;
也不绑定最终 model、参数与 hardware, 开发者仍可声明 execution preference / constraint。

Design

Orchestrator 将 Workflow Spec 转换为 request-agnostic Logical Workflow

对应 Challenge 01



Development 输出 request-agnostic Logical Workflow, 显式呈现 executor nodes 与 data-flow edges;
最终 model、resource 与 hardware 仍由 Optimizer 统一选择。

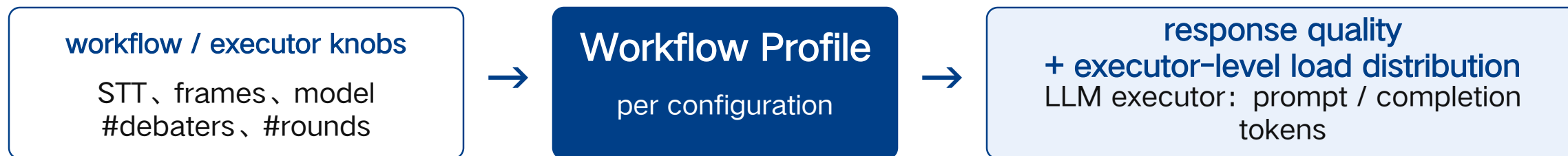
Design

两层 Profile 将 workflow 质量与负载映射为模型和硬件代价

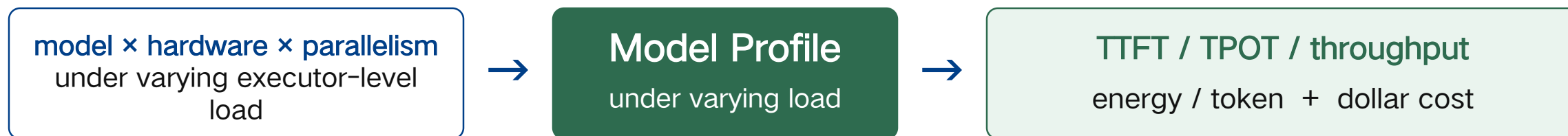
对应 Challenge 02

Offline profiling: 每个配置测量一次, 可跨 workflows 复用

01 Workflow Profile



02 Model Profile

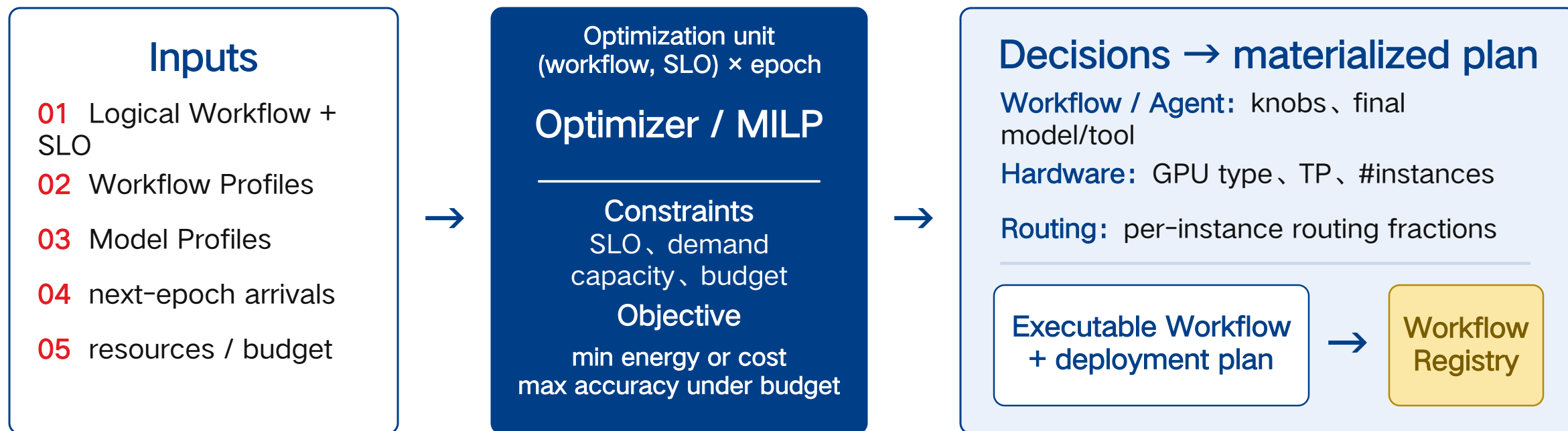


Workflow Profile 给出质量与 executor 负载; Model Profile 将负载映射为延迟、能耗与成本, 从而为 Optimizer 比较跨层候选配置提供统一依据。

Design

Optimizer 联合选择 workflow、model/tool、hardware 与 routing 配置

对应 Challenge 02



实例数按 projected peak demand 配置, routing fractions 按 average demand 优化, 以支持 multiplexing; 逐请求 dispatch 仍由 Runtime 负责。

Design

跨层优化、弹性控制与请求派发分别工作在三个时间尺度

对应 Challenge 03

01 Per epoch

Epoch | (Workflow, SLO) + Model

Optimizer

根据最新 workload 与 resource availability 重新配置 workflows
更新 Executable Workflows 写入 Workflow Registry

↑ 显著偏差触发 early re-opt

02 Continuous

Seconds – minutes | Model / Instance

Auto-Scaler Batch composition

Auto-Scaler 监控 per-model instance load
执行 reactive scale-out / scale-in, 并保留 spare resources
Batch composition: continuous, instance-level

03 Per request

Scope | Request

Runtime

workflow ID + query / inputs + SLO
→ Registry lookup → Executable Workflow
→ routing / dispatch; 逐请求路径不重新求解 MILP

Optimizer 更新 epoch 级部署计划, Auto-Scaler 吸收短期波动, Runtime 完成逐请求查询与派发;
显著 workload / resource 偏差会提前触发 re-optimization。

Evaluation

实验依次验证跨层选择、总体收益、动态适应与单请求调度

01 SLO 或优化目标变化时，最优 Workflow / Agent / Hardware 配置是否改变？

02 联合优化能否在满足 SLO 的同时减少 GPU、能耗与成本？

03 请求负载或可用资源变化时，系统能否自动重配置？

04 Workflow-stage visibility 能否改善单次 DAG 请求的硬件放置？

四个问题共同检验：Workflow visibility、Holistic control 与 Automated configuration 是否真正转化为端到端收益。

Evaluation

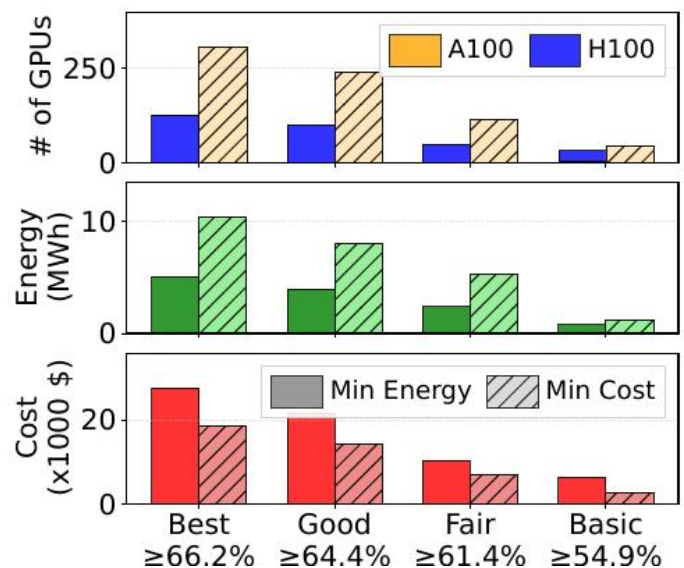
实验设置：环境、负载、调度策略与评测设置

方面	具体设置
运行环境	硬件：Azure A100 / H100 VMs，每台 VM 配置 8 × 80 GB GPU 软件：vLLM 0.9 (LLM)、speaches-ai 0.7 (STT)、OmDet (目标检测) 等
请求负载	Trace：使用 2024 年 5 月的一段 24 h Azure LLM serving trace 映射：chat 请求 → Video Q/A；coding 请求 → Code Generation 边界：没有公开生产级 Agent workflow trace，这里只复用请求到达过程
调度策略	LG (Baseline)：手工固定配置，Gemma-3-27B + A100 LG+Auto (Baseline + Auto-scaling)：LG + custom autoscaler Mkb Opt (Murakkab)：逐 Workflow - SLO 优化 (min energy / cost) Mkb Opt+Mult (Murakkab + Multiplexing)：联合优化所有 Workflow - SLO，共置并共享模型实例
评测设置	容量规划：所有策略按 P90 token-generation load 配置资源 公平比较：LG+Auto 与 Murakkab 使用相同 auto-scaling policy Murakkab 设置：optimization epoch = 60 min 观测指标：Accuracy、E2E latency、GPU 类型/数量、Energy、Cost

Evaluation

SLO 改变会触发模型、Workflow 与 Hardware 配置切换

Video Q/A | Accuracy SLO

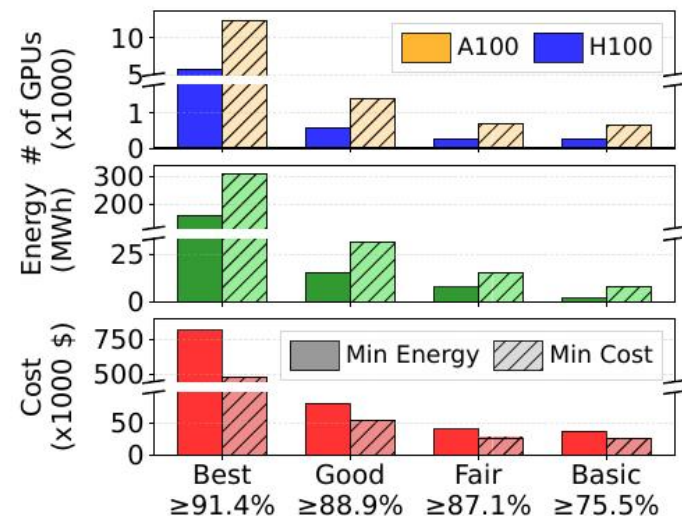


66.2% → 64.4% accuracy

5.1 → 3.9 MWh, energy 降低 23.5%

抽帧数从 10 帧 → 5 帧

Code Generation | Accuracy SLO



best → good accuracy SLO

energy $\approx 10.5\times$ 、cost $\approx 8.7\times$ lower

DeepSeek-Qwen-32B → Gemma-3-27B

只扩缩同一组模型与资源并不充分；离散的跨层配置切换构成了主要优化空间。

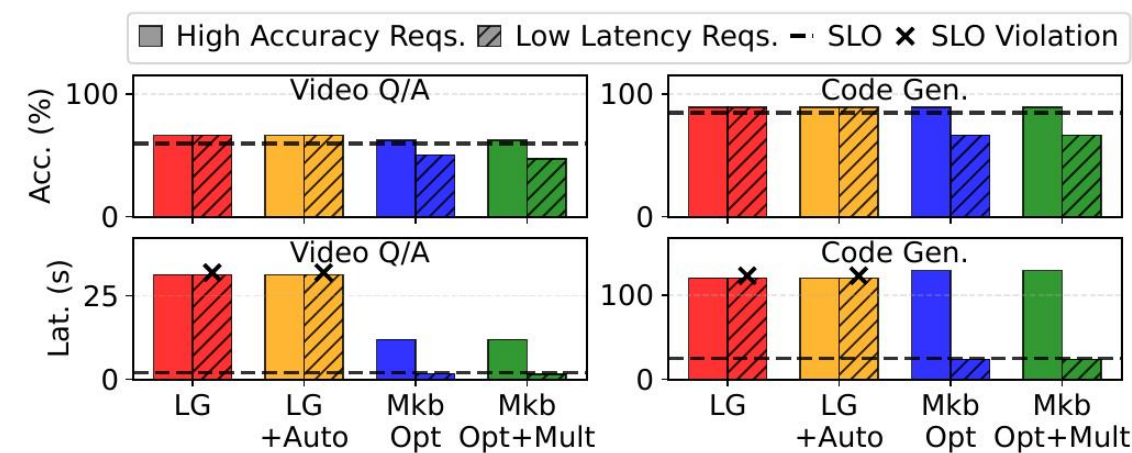
Evaluation

跨层配置贡献主要收益，Multiplexing 再带来 17% – 22% 的增量

Table 2 | 24h trace 的资源需求

Policy	# GPUs	Energy (MWh)	Cost (\$k)
LangGraph	2,568	82.1	211.7
LG + Auto	2,472	80.6	112.3
Mkb Opt	1,164	27.7	57.2
Mkb Opt + Mult	912	22.1	47.2

Figure 9b | Accuracy / Latency SLO



Mkb Opt → Mkb Opt + Mult GPU Energy Cost
 -21.6% -20.2% -17.4%

LG + Auto: 缩放固定配置 → Mkb Opt: 按 (workflow, SLO) 重选配置 → + Mult: 跨 workflow 共置与实例共享

Murakkab 不只减少资源，还利用每个请求的 SLO 选择不同配置；固定配置在低延迟请求上出现 SLO violation。

Evaluation

负载与资源供给变化时，系统会重选实例与 GPU 组合

Figure 11 | Load、实例数与 GPU allocation 随时间变化

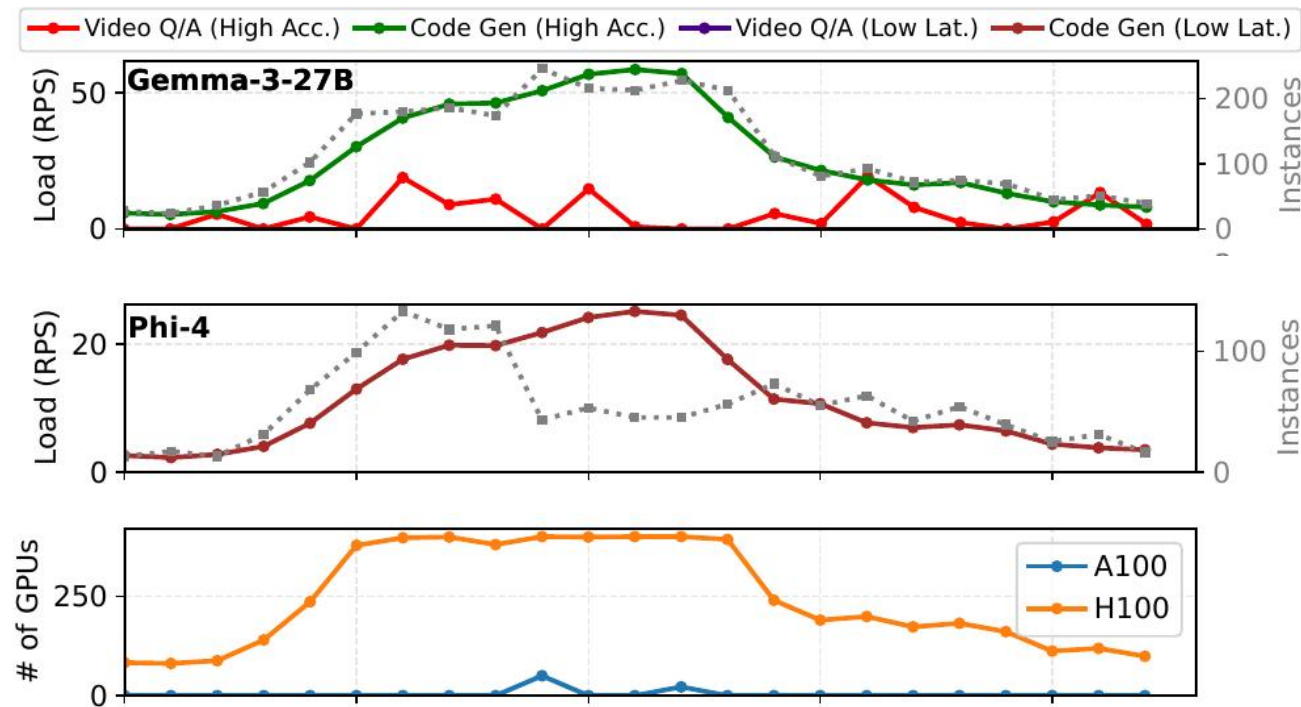


Table 3 | Energy objective 的两个端点

	H100=0	H100=500
Allocated A100	1,292	0
Allocated H100	0	495
Energy (MWh)	24.7	11.0
Cost (\$k)	55.7	75.4

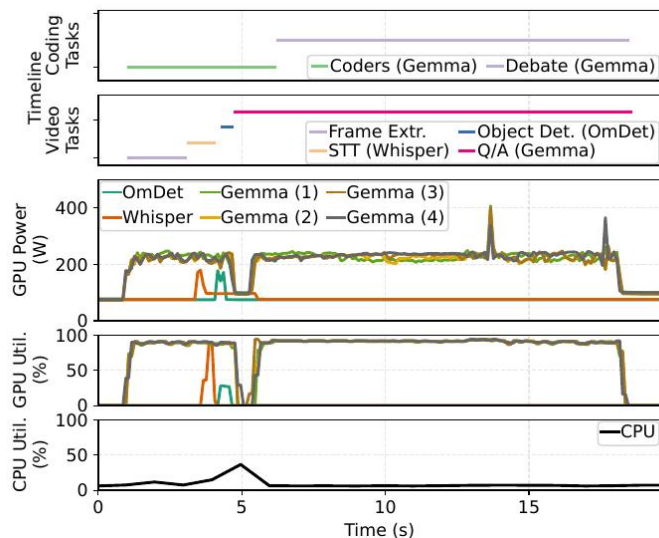
- 01 每个 optimization epoch 调整 model instances
- 02 随可用资源变化重选 A100 / H100 allocation
- 03 有剩余 capacity 时进行跨 workflow multiplexing

自动重配置服从指定优化目标：相同 SLO 下，更多 H100 使 Energy 从 24.7 降至 11.0 MWh；由于目标是 min energy，Cost 从 \$55.7k 上升到 \$75.4k。

Evaluation

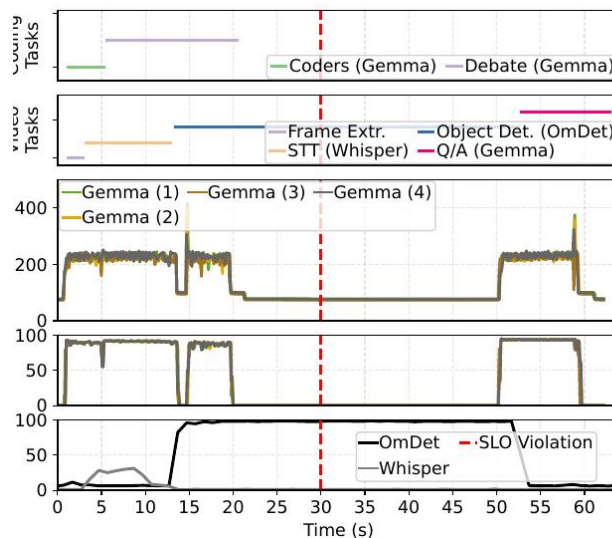
同一个 30 秒 DAG, 最省卡的可行方案并不是全部上 GPU

请求: 并行执行 Video Q/A 与 Code Generation, latency SLO = 30 s



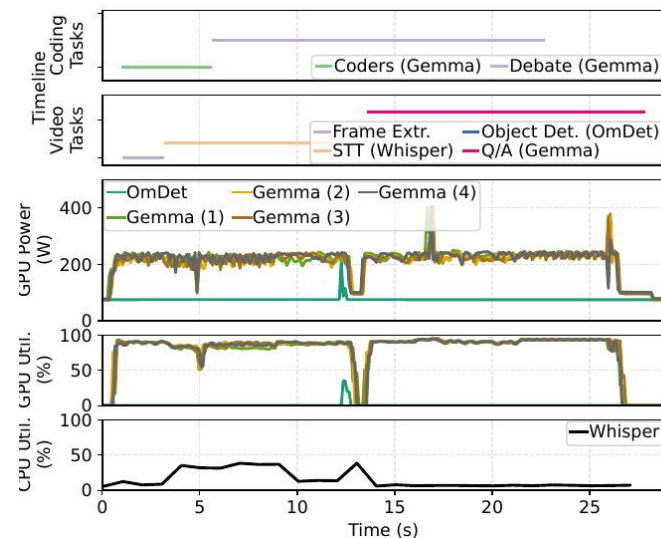
(a) 6x A100, 满足 SLO

OmDet: GPU; Whisper: GPU



(b) 4x A100, 违反 SLO

OmDet: CPU; Whisper: CPU



(c) 5x A100, 满足 SLO

OmDet: GPU; Whisper: CPU

(a) 与 (c) 都满足 30 秒 SLO; Murakkab 选择 (c), 在保持 response quality 的同时以 5 张 A100 最小化能耗, 并比全 GPU 方案少用一张卡。

文章有什么问题，基于这篇 Paper 还能做什么优化？

1

Profile 误差何时会改变 SLO 可行性判断？

论文用一天的云端模型服务请求轨迹近似工作流到达，并仅在相近任务上进行迁移验证。下一步应采集真实执行路径、生成长度和工具时延，测出何种误差会导致方案误选，并据此校准或回退。

2

求解结果能否赶在负载变化前生效？

论文把 Gurobi 求解上限设为 300 秒，却没有报告 workflow 和候选配置扩张后的求解时延、最优性差距以及方案过期率。还需要给出规模增长曲线，并尝试用上一轮结果热启动，或者只重算受影响的部分。

3

新配置带来的节省，能否抵消切换开销？

论文将新实例的准备时间统一设为 20 分钟，并用缓冲成本表示切换期间的额外资源，但不同模型和硬件的加载、预热、流量迁移与失败回退成本并未实测。下一步应把具体切换成本加入优化目标，只有新方案在预计有效期内的累计节省高于切换开销时，才执行重配置。

基于这篇 Paper，我想继续研究什么？

1

单设备端侧的跨层配置选择

先从单个设备或受控边缘节点入手，把模型版本、量化方式、工具开关以及 CPU、NPU、GPU 映射放进同一候选空间。在准确率、时延、能耗、内存和温度约束下，验证联合选择相较固定配置能否带来稳定收益。

2

以 Workflow Step 为粒度的跨端放置

不再把整条 workflow 或整个模型作为单一卸载单元，而是决定每个 step 留在本机、另一台设备、边缘节点还是云端。例如，将隐私敏感和交互关键的阶段留在本地，把重推理交给其他节点，并比较网络波动、传输开销、设备异构和节点失效下的端到端收益。

3

真实负载下的 Profile 校准与回退

持续采集实际分支路径、生成长度、工具时延和设备状态，对比 Profile 与真实运行结果。当偏差已经影响配置选择或 SLO 判断时进行在线更新；当观测不足或不确定性较高时，则保留安全余量或回退到稳定配置。



Q & A

Presenter: Yunpeng Xu

2026.7.27